



Microsoft DotNet

Free, Open-Source, Cross-Platform Framework



Day 16-17: SQL Server T-SQL Assignment

Agenda:

- Insurance Policy Management System using T-SQL Programming
- Server/Connection: SQL Server LocalDB
- Database Name: InsuranceDB
- IDE: SQL Server Management Studio (SSMS)

Learning Objectives:

After completing this assignment trainees should be able to

- Create databases and schemas
- Create tables with PK/FK constraints and insert sample data
- Write Stored Procedures
- Create User Defined Functions
- Write DML Triggers
- Work with SQL Cursors
- Handle Transactions
- Handle Exceptions using TRY...CATCH
- Generate Business Reports

Business Scenario:

ABC Insurance Ltd wants to develop a small database application for managing customers and their insurance policies. The company needs to:

- Store customer details
- Store policy information
- Store agent information
- Record premium payments
- Record claims
- Generate reports

Task 1:

- Create Database
- Create Following Tables
 1. **Customer:** CustomerId, FirstName, LastName, DOB, Gender, Phone, City
 2. **Agent:** AgentId, AgentName, Branch, ExperienceYears
 3. **Policy:** PolicyId, PolicyName, PolicyType, PremiumAmount, CoverageAmount, PolicyTermYears
 4. **CustomerPolicy:** CustomerPolicyId, CustomerId, PolicyId, AgentId, PurchaseDate, Status
 5. **PremiumPayment:** PaymentId, CustomerPolicyId, PaymentDate, Amount, PaymentMode
 6. **Claim:** ClaimId, CustomerPolicyId, ClaimDate, ClaimAmount, ClaimStatus



Microsoft DotNet

Free, Open-Source, Cross-Platform Framework



- **Insert**
 - 15 Customers
 - 6 Agents
 - 8 Policies
 - 20 Customer Policies
 - 50 Premium Payments
 - 8 Claims

Task 2: Stored Procedures

- **Stored Procedure 1:** To Add Customer
 - Input: Name, DOB, Gender, Phone, City
 - Output: Customer Added Successfully
- **Stored Procedure 2:** Assign Policy to Customer
 - Input: CustomerId, PolicyId, AgentId
 - Should: Insert CustomerPolicy and Default Status = Active
- **Stored Procedure 3:** Pay Premium
 - Input: CustomerPolicyId, Amount, PaymentMode
 - Should: Insert into PremiumPayment
- **Stored Procedure 4:** Register Claim
 - Input: CustomerPolicyId, ClaimAmount
 - Status: Pending
- **Stored Procedure 5:** Customer Policy Summary
 - Input: CustomerId
 - Output: Customer, Policy, Premium, Payments, Claim Status

Task 3: Functions

- **Scalar Function:** Calculate Annual Premium
 - Input: PolicyId
 - Return: Monthly Premium
- **Table Valued Function:** Return all Active Policies
 - Input: City
 - Output: Customer, Policy and Premium

Task 4: Triggers

- **Trigger 1:** After Insert on PremiumPayment
 - Business Rule Maintain audit table with PaymentAudit
 - Columns: AuditId, PaymentId, Action, ActionDate
- **Trigger 2:** Instead Of Delete in Customer table
 - Business Rule: Customer cannot be deleted if any active policy exists.

Task 5: Cursors

- **Cursor 1:** Generate Renewal Reminder
 - Cursor should Read every Active Policy
 - Print 'Dear Amit, Your Health Policy expires in 30 days.'



Microsoft DotNet



Free, Open-Source, Cross-Platform Framework

- **Cursor 2:** Generate Agent Performance
 - For every Agent print: Agent Name, Total Policies Sold, Total Premium Collected

Task 6: Exception Handling

- Write one Stored Procedure: Transfer Claim Amount, If Claim Amount <0 Raise Error Else Update claim. Use TRY...CATCH.

Task 7: Transaction Example

- Create a stored procedure named PurchasePolicy. This procedure should maintain SQL Transactions: Insert CustomerPolicy and Insert Premium Payment, If either fails, Rollback Else Commit transaction.

Task 8: Business Report

- Create one Stored Procedure: sp_GenerateBranchBusinessReport
 - Output as format below and sorted by Premium Collected DESC

Branch	Policies Sold	Premium Collected	Claims	Customers
--------	---------------	-------------------	--------	-----------

Expected Deliverables

By the end of the lab, each trainee should submit:

- A single .sql script that creates the database, schema, tables, constraints, and sample data.
- Five stored procedures implementing the required business operations.
- Two user-defined functions (one scalar and one table-valued).
- Two DML triggers.
- Two cursor-based programs.
- One transaction example using BEGIN TRANSACTION, COMMIT, and ROLLBACK.
- One exception-handling example using TRY...CATCH and THROW (or a constraint-generated error).
- One business report stored procedure.
- Screenshots showing successful execution and sample outputs.

Extra Tasks (optional)

- Top 5 Customers by Premium
- Highest Claim Paid
- City-wise Business
- Policy-wise Collection
- Monthly Premium Collection
- Agent Ranking

Entity Framework is the bridge, but T-SQL is the bedrock